VGTO: Visibility-Guided Topological Trajectory Optimization in Polyhedral Environments

Peiyu Cui, Hao Zhang, and Jian Sun, *Senior Member, IEEE*

***Abstract*—Planning safe and efficient flight trajectories for autonomous aerial robots in complex and unknown environments remains a challenging problem. Achieving high-quality local replanning is particularly difficult due to the inherent trade-off between computational efficiency and trajectory optimality, as well as the susceptibility of non-convex gradient-based methods to local minima. In this paper, we present VGTO, a visibility-guided topological trajectory optimization framework for real-time aerial navigation. By leveraging a lightweight polyhedral map, the proposed method eliminates the reliance on computationally expensive map representations, enabling efficient online replanning while preserving trajectory quality. Specifically, a visibility-guided topological path searching method is developed to directly generate diverse initialization paths without constructing an explicit topological graph. In addition, a lightweight obstacle distance and gradient evaluation method is introduced to enable efficient collision avoidance directly on the polyhedral map. Extensive simulations and real-world experiments demonstrate that the proposed method achieves a better balance between computational efficiency and trajectory quality, consistently improving replanning success rate and trajectory smoothness while reducing computational overhead.**

***Index Terms*—Aerial robots, perception and autonomy, collision avoidance, motion and path planning.**

I. Introduction

PLANNING safe and efficient flight trajectories is crucial for autonomous aerial navigation [1], [2], [3]. It is widely desired that aerial robots navigate in complex and unknown environments in real time while maintaining trajectory smoothness, safety, and maneuverability. However, balancing real-time performance and trajectory optimality in local replanning remains a challenging problem.

Gradient-based trajectory optimization (GTO) methods [1], [2], [3], [4] formulate replanning process as a nonlinear optimization problem, enabling locally infeasible trajectories to be efficiently refined into dynamically feasible ones that consider safety and smoothness, thus showing strong potential. However, practical trajectory optimization formulations are often non-convex, and collision-avoidance terms can further increase their non-convexity and sensitivity to trajectory initialization, potentially causing gradient-based methods to converge to local minima. Existing works [5], [6], [7] address this issue using a two-stage paradigm consisting of front-end topological path generation and back-end parallel trajectory optimization. Nevertheless, most methods rely on explicit construction of a topological graph to obtain initial topological paths, and the back-end optimization typically depends on an Occupancy Grid Map (OGM) or Euclidean Signed Distance Field (ESDF) map to provide collision gradients, which limits the real-time performance of replanning.

Despite extensive efforts to address the local minima, three major challenges remain: **1) Limited real-time performance in topological path generation:** Existing methods typically require explicit construction of a topological graph, which is computationally expensive and limits real-time performance. **2) Dependence on computationally expensive map representations:** GTO typically relies on OGM or ESDF maps, resulting in high memory consumption and slow map update. **3) Lack of lightweight collision avoidance gradient evaluation:** There is a lack of a lightweight and robust obstacle gradient evaluation method that eliminates dependence on pre-computed maps and supports efficient online computation.

To address these challenges, we propose **VGTO**, a **V**isibility-**G**uided topological **T**rajectory **O**ptimization framework. The proposed VGTO navigation framework is depicted in Fig. 1. First, topological initialization paths are directly generated based on the polyhedral map without explicit construction of a topological graph. Their non-homotopic property and distance optimality are theoretically established in 2-D environments, while in 3-D environments, the generated paths provide effective initializations with diverse visibility deformation structures. Second, obstacle gradients are directly evaluated on the compact polyhedral map, eliminating the need for OGM or ESDF map updates. We evaluate the method through extensive simulations and real-world experiments. The results demonstrate that the proposed framework outperforms existing topological planning methods in replanning success rate, trajectory optimality, and computational efficiency. The main contributions of this work are summarized as follows:

1) A visibility-guided topological path searching algorithm is proposed to efficiently generate diverse topological initialization paths, without the need for explicit construction of a topological graph.
2) A novel obstacle avoidance evaluation method is developed, which enables direct gradient computation on a lightweight polyhedral map, eliminating the reliance on OGM or ESDF map representations.
3) Extensive simulations and real-world experiments are conducted to demonstrate improvements in replanning

Received 30 April 2026; accepted 6 September 2026. Date of publication 16 September 2026; date of current version 25 September 2026. This article was recommended for publication by Associate Editor G. Li and Editor G. Loianno upon evaluation of the reviewers' comments. This work was supported by the Fundamental Research Funds for the Central Universities under Grant 22120230311. *(Corresponding author: Hao Zhang.)*

Peiyu Cui and Hao Zhang are with the College of Electronic and Information Engineering, Tongji University, Shanghai 200092, China (e-mail: Cui_Peiyu@tongji.edu.cn; zhang_hao@tongji.edu.cn).

Jian Sun is with the College of Transportation, Tongji University, Shanghai 201804, China (e-mail: sunjian@tongji.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/LRA.2026.3734891, provided by the authors.

Digital Object Identifier 10.1109/LRA.2026.3734891

2377-3766 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies.
Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS. Downloaded on October 24,2026 at 03:43:24 UTC from IEEE Xplore. Restrictions apply

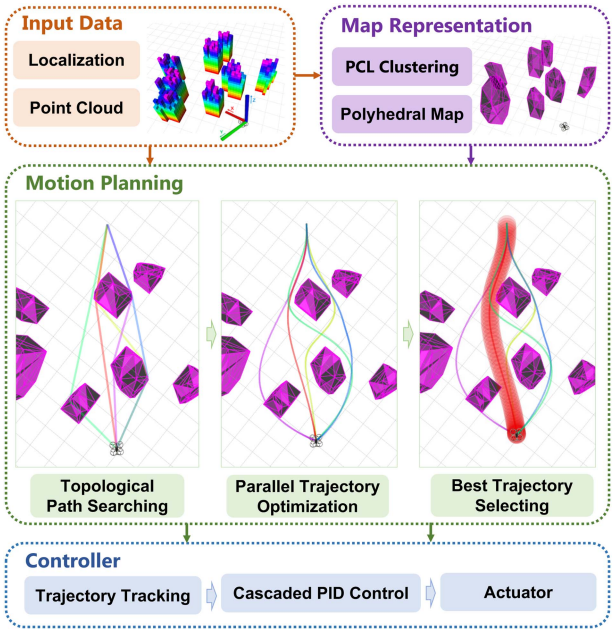


Fig. 1. Overview of the proposed online replanning framework.

success rate, optimality, and computational efficiency, compared with existing topological planners.

II. Related Work

A. Topological Path Generation

Topological path generation aims to provide diverse initial trajectories by exploring different homotopy classes, which is critical for avoiding local minima in trajectory optimization. Existing topological path generation methods can be broadly categorized into search-based and sampling-based methods. Search-based methods typically require constructing an explicit topological graph, which is augmented using topological invariants [8], [9]. Graph search algorithms [10] are then applied to extract multiple non-homotopic paths. In [8], an H-signature augmented visibility graph is constructed to ensure path optimality and topological completeness. However, the computation of the augmented visibility graph is expensive and generally unsuitable for online applications. In [11], a generalized Voronoi graph (GVG) is constructed and augmented with H-signatures to obtain different topological paths, but it also suffers from high computational cost. To improve efficiency, [12] proposes the enveloped generalized Voronoi graph (EGVG) method, which enforces the exploration of multiple homotopy classes by disabling key vertices after each iteration. However, paths generated from the Voronoi graph are generally not distance-optimal. Sampling-based methods are represented by probabilistic roadmaps (PRM) [13], which construct a roadmap by random sampling and connecting nodes with collision-free straight-line edges. Non-homotopic paths are then obtained by filtering trajectories based on their homotopy classes, but the computation of topological invariants is time-consuming. In [14] and [7], a visibility probabilistic roadmap (Visibility-PRM) is proposed, where the uniform visibility deformation (UVD) is introduced to detect homotopy classes. This reduces the cost of homotopy filtering, but its performance degrades in cluttered environments. Different from the above methods, our approach does not require constructing an explicit topological graph. Instead, we extract topological branching points within local visibility regions to guide the search process, significantly reducing the online computational cost.

B. Gradient-Based Trajectory Optimization

GTO formulates trajectory generation as an unconstrained nonlinear optimization problem. In [15], the ESDF is introduced to evaluate obstacle gradients, and covariant gradient descent is used to minimize trajectory smoothness and collision cost. However, ESDF construction is computationally expensive and typically requires offline processing. In [16] and [17], incremental online ESDF construction is achieved. However, it still requires a high update frequency (up to 50 Hz), and the map utilization in trajectory optimization remains inefficient. To eliminate the dependence on ESDF, [4] and [2] estimate obstacle distance and gradient information directly from OGM. However, these methods still require online OGM construction, and the resulting trajectories are prone to local minima. Zhou et al. [7] incorporate topological paths into the optimization process, which effectively guides the optimizer to escape infeasible local minima. However, it still relies on ESDF to evaluate obstacle distance and gradient information. Cui et al. [18] propose a lightweight polyhedral map representation and demonstrate its advantages over OGM and ESDF in terms of memory efficiency and computational cost. However, the distance and gradient information for polyhedral obstacles are not explicitly formulated. Differentiable approximations of polyhedral distance functions have been investigated in related fields. Deng et al. [19] use log-sum-exp (LSE) to smooth the face-wise maximum of a convex polyhedron for learnable convex decomposition. Liu et al. [20] adopt a similar formulation for star-convex visibility constraints, while Han et al. [21] handle static obstacles using safe driving corridors and apply an LSE-smoothed max–min distance lower bound to convex dynamic obstacle polygons. Although these studies establish LSE as a standard smoothing tool, they have not investigated a unified representation capable of simultaneously handling both convex and non-convex obstacles in polyhedral environments. The most related work to ours is [7]. We follow the paradigm of topology-guided trajectory optimization. However, different from existing methods, we redesign both the topological path search and the obstacle distance and gradient evaluation based on a lightweight polyhedral map. This enables our method to further improve optimality, while achieving higher computational efficiency.

III. Visibility-Guided Topological Path Searching

In this section, we propose a visibility-guided topological path searching method for computing multiple topological paths in polyhedral environments. Visibility regions define a topology-aware search space in which topological branching points guide node exploration. We further provide an analysis of the distance optimality of the proposed method.

A. Preliminaries and Definitions

Definition 1 (Polyhedral Map): A polyhedral map $\mathcal{M}$ denotes the workspace of the environment and contains a set of polyhedral obstacles $\mathcal{O} \subset \mathcal{M}$. Each obstacle is modeled as a convex

Authorized licensed use limited to: NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS. Downloaded on October 24,2026 at 03:43:24 UTC from IEEE Xplore. Restrictions apply

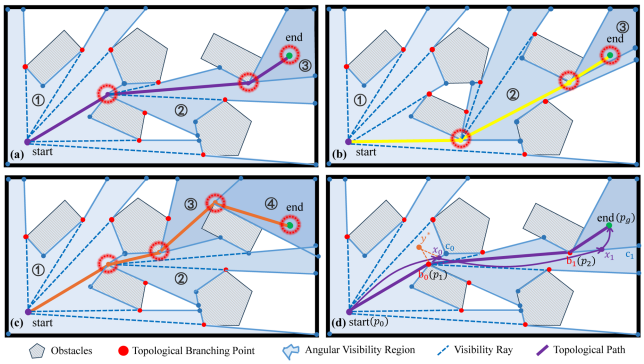


Fig. 2. An illustration of visibility-guided topological path generation. Red nodes denote topological branching points, and dashed circles indicate the selected optimal node at each search step. (a)–(c) Topological branching points are extracted based on angular visibility regions to guide the exploration of different homotopy classes. (d) Auxiliary figure for the proof of Theorem 1.

polyhedron, while non-convex obstacles are approximated by a finite union of convex polyhedra via convex decomposition.

According to [18], each convex polyhedron admits both vertex (V-) and hyperplane (H-) representations. In the V-representation, a convex polyhedron is characterized by a finite vertex set $\boldsymbol{\xi} = \{\mathbf{v}_1, \ldots, \mathbf{v}_p\} \subset \mathbb{R}^n$. Equivalently, the same polyhedron can be expressed in H-representation as

$$\mathcal{O}^{\mathcal{H}} = \left\{ \mathbf{x} \in \mathbb{R}^n \;\middle|\; \mathbf{A}\mathbf{x} \leq \mathbf{b} \right\}, \tag{1}$$

where each row $\mathbf{a}_i^\top$ of $\mathbf{A} \in \mathbb{R}^{m \times n}$ together with $b_i \in \mathbb{R}$ defines a linear half-space. When $n = 2$, each polyhedron reduces to a polygon, and $\mathcal{M}$ correspondingly becomes a polygonal map. For clarity, the path searching method and its theoretical analysis of homotopy properties and distance optimality are restricted to the 2-D polygonal case. In 3-D environments, the generated paths are instead distinguished by visibility deformation, as further clarified in Remark 1.

Definition 2 (Visibility Region): Given a polygonal map $\mathcal{M}$ with obstacle set $\mathcal{O}$ and a point $p \in \mathcal{M} \setminus \mathcal{O}$, the visibility region of p is defined as

$$\mathcal{V}(p) = \left\{ q \;\middle|\; \overline{pq} \cap \mathcal{O} = \emptyset \right\}. \tag{2}$$

Intuitively, $\mathcal{V}(p)$ consists of all points that are visible from p without being occluded by any obstacle. By default, the visibility region is defined over the full angular domain $[0, 2\pi]$. More generally, it can be restricted to an arbitrary angular interval $\Theta = [\theta_{\min}, \theta_{\max}] \subset [0, 2\pi]$, yielding

$$\mathcal{V}_\Theta(p) = \left\{ q \;\middle|\; \overline{pq} \cap \mathcal{O} = \emptyset,\ \theta_{\min} \leq \angle(q - p) \leq \theta_{\max} \right\}. \tag{3}$$

Definition 3 (Topological Branching Point): Given a point p and its visibility polygon $\mathcal{V}_\Theta(p)$, whose vertices are ordered counterclockwise along $\partial\mathcal{V}_\Theta(p)$, a vertex $b_i \in \partial\mathcal{V}_\Theta(p)$ is called a topological branching point if one of its adjacent vertices, denoted by c_i, satisfies

$$c_i = p + \lambda_i(b_i - p),\ \lambda_i > 1. \tag{4}$$

Such a point induces a critical visibility ray at which feasible paths diverge into distinct topological classes around obstacles.

Definition 4 (Non-homotopic Paths [8]): Two paths $\gamma_1, \gamma_2 : [0, 1] \to \mathcal{M}$ with $\gamma_1(0) = \gamma_2(0) = p_s$ and $\gamma_1(1) = \gamma_2(1) = p_g$ are said to be non-homotopic if one cannot be continuously deformed into the other without intersecting any obstacle.

Algorithm 1: Visibility-Guided Path Searching.

Input: Map representation $\mathcal{M}$, start p_s, goal p_g, number of non-homotopic paths K
Output: K shortest non-homotopic paths $\mathcal{P}$

1 $\mathcal{Q} \leftarrow \emptyset$, $\mathcal{P} \leftarrow \emptyset$;
2 Initialize start node n_s;
3 $\mathcal{Q}$.**push**(n_s);
4 **while** $\neg\, \mathcal{Q}$.**empty**() **do**
5 $n_c \leftarrow \mathcal{Q}$.**pop**();
6 **if** $n_c == n_s$ **then**
7 $\mathcal{V}(p_c) \leftarrow$ **getVisibilityRange**$(p_s, [0, 2\pi])$;
8 **else**
9 $\mathcal{V}(p_c) \leftarrow$ **getVisibilityRange**$(p_c, \Theta(n_c))$;
10 **if isInVisibilityRange**$(p_g, \mathcal{V}(p_c))$ **then**
11 $n_g \leftarrow$ **getGoalNode**(p_g);
12 $\gamma \leftarrow$ **RetrievePath**(n_g);
13 $\mathcal{P}$.**insert**(γ);
14 **else**
15 $\mathcal{T} \leftarrow$ **getTopoBranchPoints**$(n_c, \mathcal{V}(p_c))$;
16 **foreach** $n_i \in \mathcal{T}$ **do**
17 **if ForwardDirectionCheck**(n_c, n_i) **then**
18 $\Theta(n_i) \leftarrow$ **ComputeAngle**(n_c, n_i);
19 $g(n_i), f(n_i) \leftarrow$ **ComputeCost**(n_i);
20 $\mathcal{Q}$.**push**(n_i);
21 **if** $\mathcal{P}$.**size** $\geq K$ **then**
22 **break**;
23 **return** $\mathcal{P}$

B. Topological Path Searching

We propose a visibility-guided topological path searching algorithm to compute multiple topological paths in a polygonal environment. The overall procedure is summarized in Algorithm 1, and the generation process of the non-homotopic topological paths is illustrated in Fig. 2.

The proposed algorithm maintains a priority queue $\mathcal{Q}$ for node expansion and a path set $\mathcal{P}$ for storing non-homotopic solutions. The priority queue is ordered according to the node cost $f(n_i)$. The queue is first initialized with the start node n_s (Lines 1–3). At each iteration, the current node n_c is popped from the queue (Line 5), and its visibility region $\mathcal{V}(p_c)$ is computed. For the start node, the full angular domain $[0, 2\pi]$ is considered (Line 7). For subsequent nodes, the visibility is restricted to a locally valid angular interval (Line 9). If the goal point p_g lies within the visibility region of the current node, a valid topological path γ is obtained (Lines 10–13). Otherwise, a set of topological branching points $\mathcal{T}$ is extracted from the boundary of the visibility region (Line 15). Each branching point corresponds to a distinct topological deviation caused by obstacle occlusion, as defined in Definition 3. For each branching point $n_i \in \mathcal{T}$, we apply a forward-checking strategy inspired by [13] to avoid backtracking behavior and suppress redundant revisits (Line 17).

Authorized licensed use limited to: NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS. Downloaded on October 24,2026 at 03:43:24 UTC from IEEE Xplore. Restrictions apply

Unlike [13], where forward checking is used to reduce redundancy in PRM construction, the proposed method directly prunes topological branching points, thereby reducing exploration of redundant homotopy classes and removing self-intersecting paths. Since this is a heuristic criterion, the pruning strategy may discard feasible branches. Therefore, the distance-optimality analysis in Section III-C considers the unpruned search process.

Each node is expanded by constructing a locally angular interval (Line 18). For a branching point n_i generated from the current node n_c, its feasible angular interval is defined by two rays originating from n_i: 1) the translated incoming direction from the parent node, and 2) the directions toward the endpoints of the associated obstacle critical edge. For node cost evaluation, the standard A* cost is adopted (Line 19). The search process continues until K non-homotopic paths are found or the queue becomes empty. Finally, the algorithm returns the set $\mathcal{P}$, which contains the K shortest non-homotopic paths from the start to the goal.

C. Distance Optimality

Since the heuristic pruning strategy may discard feasible branches, the properties of the paths generated by the proposed unpruned method are established in the following theorem.

Theorem 1 (Distance Optimality): The proposed unpruned method returns a set of K paths $\mathcal{P}_K = \{\gamma_1, \dots, \gamma_K\}$ in a polygonal environment $\mathcal{M}$ with the following properties:

1) For any $i \neq j$, $\mathcal{H}(\gamma_i) \neq \mathcal{H}(\gamma_j)$.
2) For every $i = 1, \dots, K$, $L(\gamma_i) = \min_{\gamma \in \mathcal{H}(\gamma_i)} L(\gamma)$.
3) The paths satisfy $L(\gamma_1) \leq L(\gamma_2) \leq \cdots \leq L(\gamma_K)$.

Proof: First, each path is associated with a distinct homotopy class, which is guaranteed by enforcing restricted angular visibility regions at topological branching points. These branching decisions correspond to obstacle-bypassing choices, ensuring that different sequences of such decisions induce non-deformable trajectories.

Second, within each homotopy class, the algorithm identifies the shortest path. This follows from the fact that shortest paths in polygonal environments are composed of obstacle vertices [22], which are explicitly incorporated as branching points. We further demonstrate that the shortest path within each homotopy class can be reconstructed using the proposed branching process, and that the restricted angular visibility regions do not eliminate any optimal solution. As illustrated in Fig. 2(d), let $\gamma^\star$(purple curved arrow) be a shortest path of a homotopy class $\mathcal{H}$, and consider the current visibility region $\mathcal{V}_{\Theta_i}(p_i)$. If $p_g \notin \mathcal{V}_{\Theta_i}(p_i)$, then $\gamma^\star$ must leave the current visibility region. Since $\gamma^\star$ is collision-free, it cannot pass through an obstacle. Therefore, it must intersect the boundary segments $\overline{b_i c_i} \subset \partial\mathcal{V}_{\Theta_i}(p_i)$. Let the intersection point be $x_i \in \overline{b_i c_i}$. Since x_i is visible from p_i and the visibility region is simply-connected, the subpath from p_i to x_i is homotopic to the straight segment $\overline{p_i x_i}$ within the visibility region. It must satisfy $\gamma^\star[p_i, x_i] = \overline{p_i x_i}$; otherwise, replacing it with the straight segment would shorten $\gamma^\star$ without changing its homotopy class. Then, p_i, b_i and x_i are collinear, with $b_i \in \overline{p_i x_i}$. It follows that $b_i \in \gamma^\star$. The algorithm generates b_i as a topological branching point and sets $p_{i+1} = b_i$. Repeating the same argument reconstructs the successive branching points of $\gamma^\star$ until the goal becomes visible.

It remains to show that the restricted angular visibility region does not eliminate any optimal continuation. Suppose, for contradiction, that the shortest representative $\gamma^\star$ leaves b_i across the forward boundary $\overline{b_i c_i}$ of $\mathcal{V}_{\Theta_{i+1}}(p_{i+1})$. To remain consistent with the current topology branch, it must subsequently return across the boundary segment $\overline{b_i c_i}$. Let $x_i \in \overline{b_i c_i}$ be its return point, and let $y^\star$ be a point on the intervening detour, as illustrated in Fig. 2(d). Then $\|b_i - y^\star\| + \|y^\star - x_i\| > \|b_i - x_i\|$. The detour $\gamma^\star[b_i, x_i]$ can therefore be replaced by $\overline{b_i x_i}$ without changing the homotopy class, producing a strictly shorter path. Hence, the shortest path cannot cross the forward boundary $\overline{b_i c_i}$ of $\mathcal{V}_{\Theta_{i+1}}(p_{i+1})$. The other boundary of $\mathcal{V}_{\Theta_{i+1}}(p_{i+1})$ is an incident obstacle edge, and crossing it would enter the obstacle. Thus, the restricted angular visibility regions do not eliminate any optimal path.

Third, the paths are returned in non-decreasing order of path length. The priority queue is ordered by the A* cost $f(n) = g(n) + h(n)$, where $g(n)$ is the accumulated path length from p_s to node n, and $h(n) = \|p_g - p(n)\|$ is the distance heuristic from n to the goal. Since this heuristic is an admissible lower bound on the remaining path length, the resulting paths are returned in non-decreasing order of their total lengths. □

Remark 1: In a 2D polygonal environment, the paths generated by our algorithm belong to different homotopy classes. In 3D environments, the same construction generates paths that are distinguished by visibility deformation classes rather than strict homotopy classes. This is because paths in 3D have greater freedom to deform, and geometrically distinct paths may therefore still belong to the same homotopy class.

IV. Gradient-Based Trajectory Optimization

A. Optimization Formulation

The topological paths generated by the front end provide useful guidance for escaping local minima. However, for real-world robotic systems, these paths are often discontinuous, thus requiring additional refinement steps to obtain dynamically feasible trajectories. Given a geometric path, we construct a time-parameterized trajectory $\tau_{\text{geo}} : [0, T] \to \mathbb{R}^3$, where T is the local reference trajectory time intercepted from the global reference trajectory. This trajectory is uniformly sampled at intervals Δt, generating a series of control points $\mathbf{Q} = \{\mathbf{Q}_0, \dots, \mathbf{Q}_N\}$, $\mathbf{Q}_i \in \mathbb{R}^3$. These control points, as optimization variables, define the discrete trajectory representation, which is then converted into a spline curve for execution. Inspired by [7], we adopt a two-stage optimization strategy to improve robustness and convergence.

Stage 1. Topology-Guided Optimization: In the first stage, discontinuities are removed while temporarily ignoring collision and dynamic feasibility constraints. The objective is to obtain a geometrically smooth trajectory that preserves the topology of the reference path:

$$\min_{\mathbf{Q}} \; J(\mathbf{Q}) = \lambda_g J_{\text{g}} + \lambda_s J_{\text{s}}. \tag{5}$$

where J_{g} encourages the trajectory to follow the topological path, and J_{s} penalizes jerk to promote smoothness. The resulting trajectory provides a topologically consistent and smooth initialization for the next stage.

Stage 2. Trajectory Refinement: Starting from the solution of Stage 1, the second stage incorporates obstacle avoidance and dynamic feasibility constraints. We observe that if the trajectory duration is fixed, trajectories initialized from different topological paths may converge to the same topology class

Authorized licensed use limited to: NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS. Downloaded on October 24,2026 at 03:43:24 UTC from IEEE Xplore. Restrictions apply

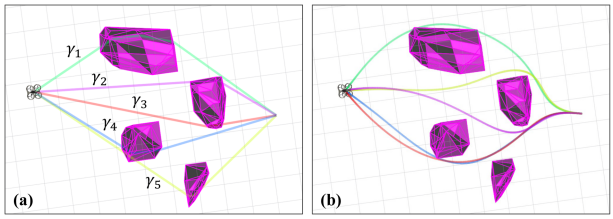


Fig. 3. Effect of fixed time allocation. (a) Initial topological paths. (b) Optimization under a shared time scale leads to topology collapse.

(as illustrated in Fig. 3). This issue arises because paths with different geometric lengths are optimized under a shared time allocation. In particular, longer paths are more likely to violate dynamic constraints under the same time scale. As a result, the optimizer tends to deform such trajectories toward dynamically feasible regions, potentially collapsing them into shorter paths that belong to the same homotopy class.

To address this issue, we further introduce trajectory duration as an optimization variable, allowing the optimizer to adapt the time allocation according to the geometric complexity of each path, thereby preserving topology diversity while ensuring dynamic feasibility. The resulting optimization problem is formulated as:

$$\min_{\mathbf{Q},T} \; J(\mathbf{Q},T) = \lambda_g J_{\mathrm{g}} + \lambda_s J_{\mathrm{s}} + \lambda_o J_{\mathrm{o}} + \lambda_t T + \lambda_v J_{\mathrm{v}} + \lambda_a J_{\mathrm{a}}. \tag{6}$$

This stage jointly optimizes trajectory duration T, safety J_{o}, and dynamic feasibility in terms of velocity J_{v} and acceleration J_{a}. The coarse-to-fine strategy improves numerical stability and reduces the likelihood of poor local minima by avoiding direct optimization from discontinuous paths.

B. Cost Function

1) Topological Path Guidance: To encourage each optimized trajectory to remain geometrically close to the original path, deviations from the sampled geometric trajectory are penalized:

$$J_{\mathrm{g}} = \sum_{i=0}^{N} \left\| \mathbf{Q}_i - \bar{\mathbf{Q}}_i \right\|^2, \tag{7}$$

where $\bar{\mathbf{Q}}_i$ are reference samples from τ_{geo}.

2) Smoothness: Trajectory smoothness is enforced via a second-order finite-difference penalty:

$$J_{\mathrm{s}} = \sum_{i=1}^{N-1} \left\| \mathbf{Q}_{i-1} - 2\mathbf{Q}_i + \mathbf{Q}_{i+1} \right\|^2. \tag{8}$$

3) Obstacle Avoidance: Polyhedral obstacle avoidance is formulated using an approximate signed distance representation similar to those in [20], [21]. Unlike these works, we extend the formulation to handle both convex and non-convex obstacles. We first consider a convex obstacle $\mathcal{O}_j$, represented in half-space form as

$$\mathcal{O}_j = \left\{ \mathbf{x} \in \mathbb{R}^3 \;\middle|\; \mathbf{a}_{j,k}^{\top}\mathbf{x} - b_{j,k} \le 0, \quad k = 1, \dots, M_j \right\}, \tag{9}$$

where $\mathbf{a}_{j,k} \in \mathbb{R}^3$ is the outward unit normal of the k-th supporting hyperplane and $b_{j,k} \in \mathbb{R}$ is its corresponding offset.

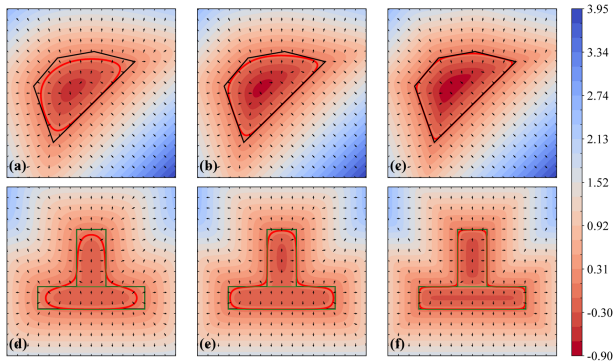


Fig. 4. LSE-based approximation of signed distance and corresponding gradient visualizations. (a)–(c) Convex obstacles with $\alpha \in \{3, 5, 10\}$; (d)–(f) Non-convex obstacles with $\alpha = \beta \in \{3, 5, 10\}$.

For a point $\mathbf{Q}_i$, its approximate signed distance to the convex polyhedron $\mathcal{O}_j$ is defined as the maximum constraint violation over its hyperplanes:

$$d_i^{\text{convex}}(\mathcal{O}_j) = \max_{1 \le k \le M_j} \left(\mathbf{a}_{j,k}^{\top}\mathbf{Q}_i - b_{j,k} \right). \tag{10}$$

For a general non-convex obstacle, $\mathcal{O}_j$ is represented as a union of L_j convex components: $\mathcal{O}_j = \bigcup_{l=1}^{L_j} \mathcal{O}_j^l$. Its approximate signed distance is then defined as

$$d_i^{\text{non-convex}}(\mathcal{O}_j) = \min_{1 \le l \le L_j} d_i^{\text{convex}}(\mathcal{O}_j^l). \tag{11}$$

Since both the $\max$ and $\min$ operators are non-differentiable, we adopt LSE approximations to obtain smooth surrogates. The convex distance is approximated as

$$\widetilde{d}_i^{\text{convex}}(\mathcal{O}_j) = \frac{1}{\alpha} \log \left(\sum_{k=1}^{M_j} e^{\alpha(\mathbf{a}_{j,k}^{\top}\mathbf{Q}_i - b_{j,k})} \right), \tag{12}$$

and the non-convex distance is approximated as

$$\widetilde{d}_i^{\text{non-convex}}(\mathcal{O}_j) = -\frac{1}{\beta} \log \left(\sum_{l=1}^{L_j} e^{-\beta \widetilde{d}_i^{\text{convex}}(\mathcal{O}_j^l)} \right), \tag{13}$$

where $\alpha > 0$ and $\beta > 0$ control the approximation accuracy, and Fig. 4 illustrates the resulting distance fields for both convex and non-convex obstacles under different parameter settings. The LSE approximation admits the following bounds:

$$d_i^{\text{convex}}(\mathcal{O}_j) \le \widetilde{d}_i^{\text{convex}}(\mathcal{O}_j) \le d_i^{\text{convex}}(\mathcal{O}_j) + \frac{\log M_j}{\alpha}, \tag{14}$$

with $\lim_{\alpha\to\infty} \widetilde{d}_i^{\text{convex}}(\mathcal{O}_j) = d_i^{\text{convex}}(\mathcal{O}_j)$. Similarly, the non-convex LSE aggregation satisfies $\lim_{\alpha,\beta\to\infty} \widetilde{d}_i^{\text{non-convex}}(\mathcal{O}_j) = d_i^{\text{non-convex}}(\mathcal{O}_j)$.

For clarity, we denote $\widetilde{d}_i$ as the distance from control point $\mathbf{Q}_i$ to its nearest obstacle, computed using Eqs. (12) and (13). Violations of a desired safety margin $d_{\mathrm{safe}} > 0$ are penalized using a hinge-quadratic function:

$$J_{\mathrm{o}} = \sum_{i=0}^{N} \mathcal{P}(\widetilde{d}_i, d_{\mathrm{safe}}), \; \mathcal{P}(x,y) = \begin{cases} (x-y)^2, & x < y, \\ 0, & x \ge y. \end{cases} \tag{15}$$

Authorized licensed use limited to: NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS. Downloaded on October 24,2026 at 03:43:24 UTC from IEEE Xplore. Restrictions apply

TABLE I
BENCHMARK COMPARISON OF TOPOLOGICAL PATH SEARCHING METHODS ACROSS DIFFERENT 2-D SCENES

Density	Method	$K=1$		$K=3$		$K=5$		$K=7$		N_{topo}/T
		$\bar{t}_c$ (ms)	$\bar{l}_p$ (m)	$\bar{t}_c$ (ms)	$\bar{l}_p$ (m)	$\bar{t}_c$ (ms)	$\bar{l}_p$ (m)	$\bar{t}_c$ (ms)	$\bar{l}_p$ (m)	
Sparse	Zhou's [7]	7.16	14.06	9.12	14.33	**10.09**	14.48	30.48	14.58	2.73
	Qi's [12]	13.91	13.96	14.43	14.30	15.74	14.71	**16.08**	15.28	2.33
	Bhattacharya's [8]	76.71	**12.76**	297.96	**12.84**	381.88	**12.87**	770.26	**12.91**	0.07
	Proposed	**1.79**	**12.76**	**8.64**	**12.84**	12.49	**12.87**	18.53	**12.91**	**3.35**
Medium	Zhou's [7]	7.74	14.27	9.62	14.55	25.24	14.68	31.81	14.89	2.13
	Qi's [12]	17.93	14.11	18.73	14.36	19.35	14.61	**20.56**	15.18	1.82
	Bhattacharya's [8]	148.52	**12.81**	536.10	**12.87**	1223.58	**12.88**	1457.73	**12.93**	0.04
	Proposed	**4.32**	**12.81**	**9.41**	**12.87**	**16.93**	**12.88**	27.26	**12.93**	**2.77**
Dense	Zhou's [7]	6.38	14.35	10.63	14.80	21.12	15.03	38.43	15.17	1.65
	Qi's [12]	25.08	14.04	25.97	14.20	26.87	14.58	29.56	15.03	1.32
	Bhattacharya's [8]	173.35	**12.81**	675.52	**12.84**	1409.82	**12.94**	2204.38	**12.97**	0.02
	Proposed	**5.31**	**12.81**	**10.26**	**12.84**	**16.84**	**12.94**	**27.34**	**12.97**	**1.96**

4) Dynamic Feasibility: The penalties on velocity and acceleration are defined as

$$J_v = \sum_{i=0}^{N-1} \left\{ \sum_{\mu\in\{x,y,z\}} P\left(v_{\max}, |v_{i,\mu}|\right) \right\}, \tag{16}$$

$$J_a = \sum_{i=0}^{N-2} \left\{ \sum_{\mu\in\{x,y,z\}} P\left(a_{\max}, |a_{i,\mu}|\right) \right\}, \tag{17}$$

where $v_{i,\mu}$ and $a_{i,\mu}$ denote the velocity and acceleration components along axis μ, respectively.

C. Trajectory Selection

To select the best trajectory for execution, each trajectory is evaluated using a lightweight cost metric that balances path length, smoothness, and consistency. Let $\mathbf{p}(t)$ denote the trajectory, and define the selection cost as follows:

$$J_{\text{select}} = w_l\, \mathcal{L}(\mathbf{p}(t)) + w_j\, \mathcal{J}(\mathbf{p}) + w_c\, C, \tag{18}$$

where $\mathcal{L}(\mathbf{p}(t))$ denotes the trajectory length, $\mathcal{J}(\mathbf{p}(t))$ measures the overall jerk magnitude, and C is a consistency term defined as

$$C = \mathbb{I}(\mathcal{H}(\mathbf{p}(t)) \neq \mathcal{H}_{\text{prev}}), \tag{19}$$

where $\mathcal{H}(\mathbf{p})$ denotes the topology class of the current trajectory, $\mathcal{H}_{\text{prev}}$ is that of the previously selected trajectory, and $\mathbb{I}(\cdot)$ is the indicator function. The trajectory with the minimum selection cost is selected for execution.

V. EXPERIMENT

A. Benchmark Comparison

1) Comparisons of the Topological Path Generation: We compare the proposed topological path generation method with three representative baselines: 1) Sampling-based method [7], which constructs a topological graph via Visibility-PRM and identifies homotopy classes using UVD-based filtering; 2) Voronoi graph-based method [12], which builds a GVG from an ESDF map; and 3) Search-based method [8], which constructs an H-signature-augmented visibility graph and performs graph search to obtain non-homotopy paths, and is generally regarded as both optimal and topologically complete.

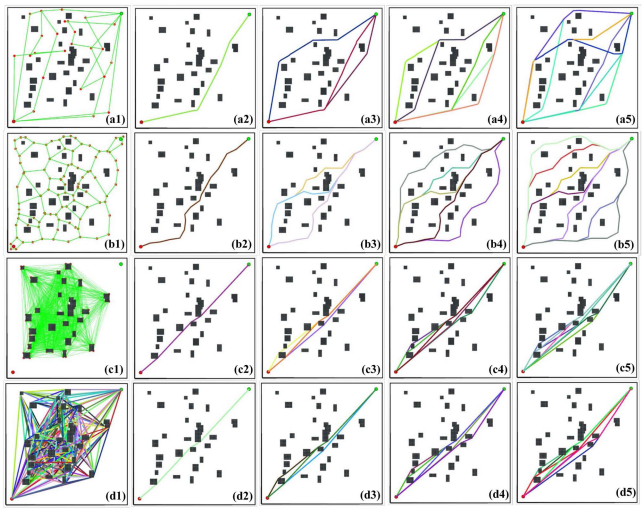


Fig. 5. Comparison of topology representations and generated paths under different methods. Subfigures (a1)– (c1) show the constructed topological graphs for the baseline methods, and (d1) shows all 25402 non-homotopic paths generated by our method. Subfigures (a2)– (a5), (b2)–(b5), (c2)–(c5), and (d2)–(d5) illustrate the results under different maximum numbers of allowed homotopy classes $K \in \{1, 3, 5, 7\}$.

We evaluate all methods in randomly generated cluttered 2-D environments with varying obstacle densities. For each environment setting, we vary the maximum number of allowed homotopy classes and record the average computation time $\bar{t}_c$ and path length $\bar{l}_p$. To further assess the ability of different methods to discover diverse topological paths, we additionally report the ratio between the number of distinct homotopy paths found N_{topo} and the computation time under a fixed time budget T. All reported results are averaged over 100 independent trials.

As illustrated in Fig. 5, the proposed method eliminates the necessity for pre-constructing explicit topological graphs. Unlike graph-dependent approaches that restrict path discovery to pre-defined connectivity structures, our method guides the path search directly through topological branching points, thereby avoiding explicit topological graph construction and improving

Authorized licensed use limited to: NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS. Downloaded on October 24,2026 at 03:43:24 UTC from IEEE Xplore. Restrictions apply

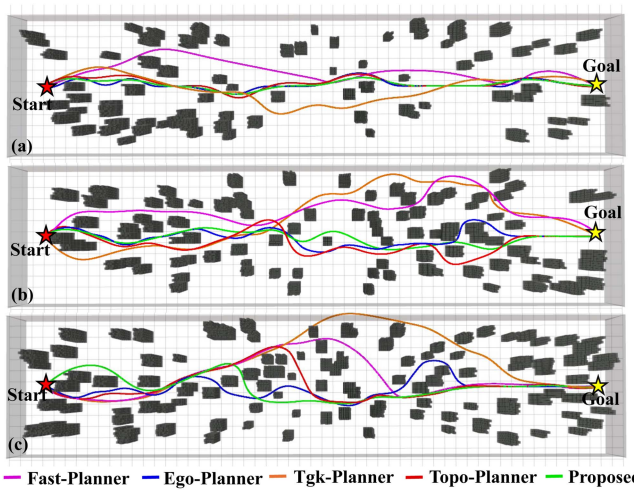


Fig. 6. Trajectory comparison in cluttered environments with varying obstacle densities. (a) Sparse scene. (b) Medium scene. (c) Dense scene.

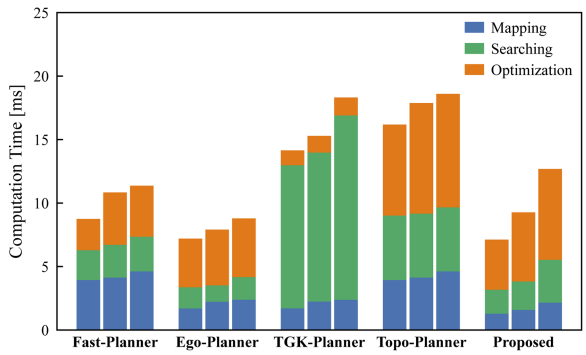


Fig. 7. Computation time breakdown of different planners under varying environment densities.

computational efficiency. The results in Table I demonstrate that the proposed method achieves the lowest computation time and comparable average path length in most settings. Notably, in dense obstacle scenarios, our method achieves approximately 80 times faster computation compared to [8]. The consistently high N_{topo}/T ratio across all scenarios further validates that our method discovers more homotopic classes more effectively under strict real-time constraints.

2) Comparisons of the Local Replanning Pipeline: As illustrated in Fig. 6, we compare the proposed planner with Fast-Planner [3], Ego-Planner [5], Tgk-Planner [6], and Topo-Planner [7] under varying obstacle densities. We report success rate η_{succ}, total computation time t_{sum}, trajectory length l_{traj}, and trajectory jerk j_{traj}.

In sparse environments, all methods achieve 100% success rate, while the proposed method generates the lowest trajectory jerk and the shortest path length, indicating smoother and more efficient motion. As the environment becomes more cluttered, baseline methods([3], [5], [6]) exhibit either degraded success rate or increased trajectory cost, whereas [7] maintains higher robustness but at the expense of higher computation time and jerk. In contrast, the proposed method consistently maintains 100% success rate across all scenarios, while achieving competitive computation time and the lowest trajectory jerk. Table II further

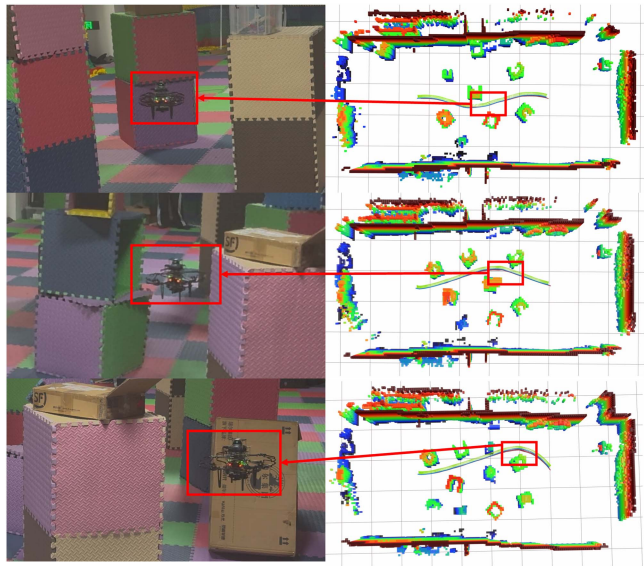


Fig. 8. Real-world flight experiments in unknown environments with different obstacle densities (from top to bottom: sparse, medium, and dense).

TABLE II
BENCHMARK COMPARISON OF TRAJECTORY PLANNERS UNDER DIFFERENT OBSTACLE DENSITIES

Density	Method	η_{succ} (%)	t_{sum} (ms)	l_{traj} (m)	j_{traj} (m^2/s^5)
Sparse	Fast-Planner [3]	**100**	8.78	51.75	130.19
	Ego-Planner [5]	**100**	7.27	50.69	155.32
	Tgk-Planner [6]	**100**	14.15	51.66	127.15
	Topo-Planner [7]	**100**	16.21	49.22	122.83
	Proposed	**100**	**7.16**	**48.55**	**106.95**
Medium	Fast-Planner [3]	88	10.88	54.83	155.54
	Ego-Planner [5]	92	**7.94**	52.54	239.49
	Tgk-Planner [6]	96	15.29	53.172	158.35
	Topo-Planner [7]	**100**	17.94	52.94	154.7
	Proposed	**100**	9.32	**49.65**	**144.64**
Dense	Fast-Planner [3]	83	11.41	53.09	193.98
	Ego-Planner [5]	89	**8.81**	**50.85**	262.34
	Tgk-Planner [6]	90	18.32	53.56	188.72
	Topo-Planner [7]	98	18.65	52.02	181.06
	Proposed	**100**	12.71	51.44	**152.39**

shows that the proposed method achieves the best or comparable trajectory length across all densities, while significantly reducing jerk, demonstrating superior trajectory quality.

Furthermore, as illustrated in Fig. 7, the computation time breakdown highlights the impact of different map representations. Ego-Planner and Tgk-Planner constructs an OGM, while Fast-Planner and Topo-Planner rely on ESDF representations with higher computational cost. In contrast, the proposed method uses a lightweight polyhedral map. The reported mapping time covers the entire process of constructing this map from the local point cloud, including downsampling, clustering, and convex-hull generation. Although Ego-Planner and Fast-Planner achieve faster computation, they are more prone to local optima, leading to inferior trajectory quality. Tgk-Planner, Topo-Planner and the proposed method improve trajectory optimality by exploring richer topological structures. However, the proposed method

Authorized licensed use limited to: NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS. Downloaded on October 24,2026 at 03:43:24 UTC from IEEE Xplore. Restrictions apply

further achieves higher efficiency, resulting in a better balance between computational cost and trajectory quality. Overall, the results indicate that the proposed planner achieves a better trade-off between efficiency and trajectory smoothness in cluttered environments.

B. Real World Experiments

To validate the proposed replanning method, real-world flight experiments are conducted in three unknown environments with sparse, medium, and dense obstacle distributions. Fig. 8 presents representative flight snapshots and the corresponding accumulated point-cloud maps and trajectories. State estimation is performed using Faster-LIO [23]. The environment is represented using PolyMap [18], while trajectory tracking is performed by a geometric controller. All modules, including perception, state estimation, mapping, planning, and control, are executed fully onboard a Jetson Orin NX, enabling fully autonomous operation without reliance on external sensing or computation infrastructure.

VI. Conclusions and Future Works

In this paper, we presented VGTO, a visibility-guided topological trajectory optimization framework for real-time aerial navigation in complex environments. The proposed method leverages a lightweight polyhedral map to avoid reliance on computationally expensive map representations while maintaining high-quality trajectory generation. Extensive simulation and real-world experiments demonstrate that the proposed approach consistently improves replanning success rate and trajectory smoothness, while significantly reducing computational overhead compared to existing methods.

A current limitation is that the gradient-based trajectory optimization requires parameter tuning. Future work will integrate our topology-guided approach with hard-constrained optimization frameworks.

References

[1] B. Zhou, F. Gao, L. Wang, C. Liu, and S. Shen, "Robust and efficient quadrotor trajectory generation for fast autonomous flight," *IEEE Robot. Automat. Lett.*, vol. 4, no. 4, pp. 3529–3536, Oct. 2019.

[2] X. Zhou, Z. Wang, H. Ye, C. Xu, and F. Gao, "EGO-planner: An ESDF-free gradient-based local planner for quadrotors," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 478–485, Apr. 2021.

[3] B. Zhou, J. Pan, F. Gao, and S. Shen, "RAPTOR: Robust and perception-aware trajectory replanning for quadrotor fast flight," *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 1992–2009, Dec. 2021.

[4] F. Gao, Y. Lin, and S. Shen, "Gradient-based online safe trajectory generation for quadrotor flight in complex environments," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2017, pp. 3681–3688.

[5] X. Zhou, J. Zhu, H. Zhou, C. Xu, and F. Gao, "EGO-Swarm: A fully autonomous and decentralized quadrotor swarm system in cluttered environments," in *Proc. 2021 IEEE Int. Conf. Robot. Automat.*, 2021, pp. 4101–4107.

[6] H. Ye, X. Zhou, Z. Wang, C. Xu, J. Chu, and F. Gao, "TGK-planner: An efficient topology guided kinodynamic planner for autonomous quadrotors," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 494–501, 2021.

[7] B. Zhou, F. Gao, J. Pan, and S. Shen, "Robust real-time UAV replanning using guided gradient-based optimization and topological paths," in *Proc. 2020 IEEE Int. Conf. Robot. Automat.*, 2020, pp. 1208–1214.

[8] S. Bhattacharya, M. Likhachev, and V. Kumar, "Topological constraints in search-based robot path planning," *Auton. Robots*, vol. 33, no. 3, pp. 273–290, 2012.

[9] M.-A. Berger, "Topological invariants in braid theory," *Lett. Math. Phys.*, vol. 55, no. 3, pp. 181–192, 2001.

[10] S.-M. LaValle, *Planning Algorithms*. Cambridge, UK: Cambridge Univ. Press, 2006.

[11] C. Rösmann, F. Hoffmann, and T. Bertram, "Integrated online trajectory planning and optimization in distinctive topologies," *Robot. Auton. Syst.*, vol. 88, pp. 142–153, 2017.

[12] H. Qi et al., "Enveloped generalized Voronoi graph and multihomotopy path planning for navigation in unknown environments," *IEEE/ASME Trans. Mechatron.*, early access, Mar. 05, 2026, doi: 10.1109/TMECH.2026.3665212.

[13] C. Rösmann, F. Hoffmann, and T. Bertram, "Planning of multiple robot trajectories in distinctive topologies," in *Proc. 2015 Eur. Conf. Mobile Robots*, 2015, pp. 1–6.

[14] T. Siméon, J.-P. Laumond, and C. Nissoux, "Visibility-based probabilistic roadmaps for motion planning," *Adv. Robot.*, vol. 14, no. 6, pp. 477–493, 2000.

[15] M. Zucker et al., "Chomp: Covariant hamiltonian optimization for motion planning," *Int. J. Robot. Res.*, vol. 32, no. 9, pp. 1164–1193, 2013.

[16] H. Oleynikova, Z. Taylor, M. Fehr, R. Siegwart, and J. Nieto, "Voxblox: Incremental 3D euclidean signed distance fields for on-board MAV planning," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2017, pp. 1366–1373.

[17] L. Han, F. Gao, B. Zhou, and S. Shen, "FIESTA: Fast incremental euclidean distance fields for online motion planning of aerial robots," in *Proc. 2019 IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 4423–4430.

[18] P. Cui, H. Zhang, Z. Du, and H. Yan, "Polymap: A lightweight polyhedral map based on approximate convex decomposition for aerial robot motion planning," *IEEE Robot. Automat. Lett.*, vol. 11, no. 3, pp. 3055–3062, 2026.

[19] B. Deng, K. Genova, S. Yazdani, S. Bouaziz, G. Hinton, and A. Tagliasacchi, "CVXNet: Learnable convex decomposition," in *Proc. 2020 IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 31–41.

[20] T. Liu et al., "Star-convex constrained optimization for visibility planning with application to aerial inspection," in *Proc. 2022 Int. Conf. Robot. Automat.*, 2022, pp. 7861–7867.

[21] Z. Han et al., "An efficient spatial-temporal trajectory planner for autonomous vehicles in unstructured environments," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 2, pp. 1797–1814, 2024.

[22] T. Lozano-Pérez and M.-A. Wesley, "An algorithm for planning collision-free paths among polyhedral obstacles," *Commun. ACM*, vol. 22, no. 10, pp. 560–570, 1979.

[23] C. Bai, T. Xiao, Y. Chen, H. Wang, F. Zhang, and X. Gao, "Faster-LIO: Lightweight tightly coupled LiDAR-inertial odometry using parallel sparse incremental voxels," *IEEE Robot. Automat. Lett.*, vol. 7, no. 2, pp. 4861–4868, Apr. 2022.

Authorized licensed use limited to: NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS. Downloaded on October 24,2026 at 03:43:24 UTC from IEEE Xplore. Restrictions apply